dorodango

Draw squircles in Typst with tunable corner smoothing

Contents

Introduction

A squircle is a rounded rectangle whose corners blend smoothly into its straight edges. Unlike a standard rounded rectangle, which transitions abruptly from a straight edge to a circular arc, a squircle has a gradual change in curvature around each corner. As a result, a squircle reads as one unified shape, rather than a square that has simply had its corners clipped or rounded off.

Rectangle Squircle

`dorodango` draws squircles by placing two cubic Bézier transitions between the circular portion of each corner and its adjoining straight edges[1]. The Bézier control points determine the extent and curvature profile of these transitions and, therefore, the smoothness of the corners. When the adjoining straight edges do not provide enough room for the full transitions, the transitions are normally shortened to fit. Alternatively, the Bézier handles can be shortened, compressing the corner while preserving the intended smoothness[2].

API reference

squircle

Draws a rectangle with smoothly rounded corners.

Use `squircle` like `rect` when you want softer, more continuous corner transitions. With `smoothing: 0%`, it has the same geometry as a rounded `rect`.

Parameters

```
squircle(
  width: auto length ratio relative,
  height: auto length ratio relative fraction,
  fill: none color gradient tiling,
  stroke: auto none length color gradient tiling stroke dictionary,
  radius: length ratio relative dictionary,
  inset: length ratio relative dictionary,
  outset: length ratio relative dictionary,
  smoothing: length ratio relative dictionary,
  preserve-smoothing: bool,
  ..body: none content str symbol
) -> content
```

width `auto` or `length` or `ratio` or `relative`

The squircle's width, relative to its parent container.

Default: `auto`

[1] Daniel Furse, "Desperately seeking squircles"
[2] phamfoo/figma-squircle implementation

height `auto` or `length` or `ratio` or `relative` or `fraction`

The squircle's height, relative to its parent container.

Default: `auto`

fill `none` or `color` or `gradient` or `tiling`

How to fill the squircle.

When setting a fill, the default stroke disappears. To create a squircle with both fill and stroke, you have to configure both.

Default: `none`

stroke `auto` or `none` or `length` or `color` or `gradient` or `tiling` or `stroke` or `dictionary`

How to stroke the squircle. This can be:

- `none` to disable stroking.
- `auto` for a stroke of `1pt + black` if and only if no fill is given.
- Any kind of stroke.
- A dictionary describing the stroke for each side individually. The dictionary can contain the following keys in order of precedence:
  - `top`: The top stroke.
  - `right`: The right stroke.
  - `bottom`: The bottom stroke.
  - `left`: The left stroke.
  - `x`: The left and right stroke.
  - `y`: The top and bottom stroke.
  - `rest`: The stroke on all sides except those for which the dictionary explicitly sets a stroke.

All keys are optional. Omitted sides are not stroked.

Default: `auto`

radius `length` or `ratio` or `relative` or `dictionary`

How much to round the squircle's corners, relative to the minimum of the width and height divided by two. This can be:

- A relative length for a uniform corner radius.
- A dictionary describing the radius for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner radius.
  - `top-right`: The top-right corner radius.
  - `bottom-right`: The bottom-right corner radius.
  - `bottom-left`: The bottom-left corner radius.
  - `left`: The top-left and bottom-left corner radii.
  - `top`: The top-left and top-right corner radii.
  - `right`: The top-right and bottom-right corner radii.
  - `bottom`: The bottom-left and bottom-right corner radii.
  - `rest`: The radii for all corners except those for which the dictionary explicitly sets a radius.

Default: `0pt`

inset `length` or `ratio` or `relative` or `dictionary`

How much to pad the squircle's content. See `box`'s `inset` parameter for more details.

Default: `5pt`

outset `length` or `ratio` or `relative` or `dictionary`

How much to expand the squircle's size without affecting the layout. See `box`'s `outset` parameter for more details.

Default: `0pt`

smoothing `length` or `ratio` or `relative` or `dictionary`

How strongly to smooth the squircle's corners. Smoothing replaces the ends of each circular corner arc with Bézier transitions whose curvature gradually changes between the straight edges and the arc. This can be:

- A relative length for uniform corner smoothing.
- A dictionary describing the smoothing for each corner individually. The dictionary can contain the following keys in order of precedence:
  - `top-left`: The top-left corner smoothing.
  - `top-right`: The top-right corner smoothing.
  - `bottom-right`: The bottom-right corner smoothing.
  - `bottom-left`: The bottom-left corner smoothing.
  - `left`: The top-left and bottom-left corner smoothing.
  - `top`: The top-left and top-right corner smoothing.
  - `right`: The top-right and bottom-right corner smoothing.
  - `bottom`: The bottom-left and bottom-right corner smoothing.
  - `rest`: The smoothing for all corners except those for which the dictionary explicitly sets smoothing.

At `0%`, the corner is a quarter circle matching a rounded `rect`. At `100%`, it is two Bézier transitions with no circular arc.

Default: `60%`

preserve-smoothing `bool`

Whether to preserve the requested smoothing when it exceeds a corner's available space. This has no effect when the smoothing already fits.

If `false`, smoothing is reduced to fit, so further increases may not change the corner. If `true`, its arc and smoothing angles are retained by shortening Bézier handles, which can make the corner look compressed.

Default: `false`

..body `none` or `content` or `str` or `symbol`

The content to place into the squircle. Strings and symbols are converted to content.

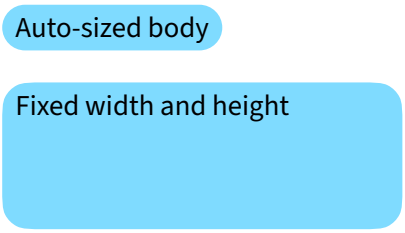
When this is omitted, the squircle takes on a default size of at most `45pt` by `30pt`.

Examples

Size and body

`width` and `height` set the squircle's layout size. When both are `auto`, the squircle takes its size from the positional body passed in square brackets, as the first shape below illustrates, while the second uses fixed dimensions.

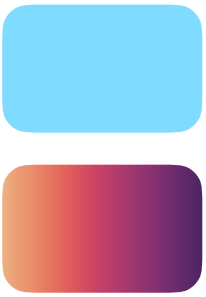
```
#grid(
  rows: 2,
  gutter: 12pt,
  squircle(radius: 10pt, fill: aqua)[Auto-sized
body],
  squircle(
    width: 150pt,
    height: 55pt,
    radius: 10pt,
    fill: aqua,
  )[Fixed width and height],
)
```



Fill

`fill` sets the squircle's interior paint. In the example below, the first shape uses a solid color and the second uses a linear gradient.

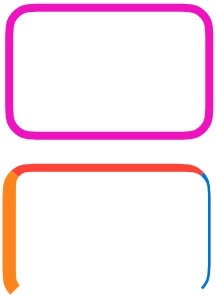
```
#grid(
  rows: 2,
  gutter: 12pt,
  squircle(width: 75pt, height: 48pt, radius:
10pt, fill: aqua),
  squircle(width: 75pt, height: 48pt, radius:
10pt, fill: gradient.linear(..color.map.flare)),
)
```



Stroke

`stroke` draws the outline. In the example below, the first shape uses one stroke on every edge, while the second assigns each edge its own stroke.

```
#grid(
  rows: 2,
  gutter: 12pt,
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 10pt,
    stroke: 3pt + fuchsia,
  ),
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 10pt,
    stroke: (top: 3pt + red, bottom: none, right:
blue, left: 5pt + orange),
  ),
)
```



Radius

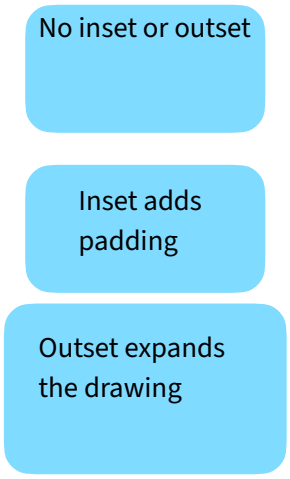
`radius` controls corner rounding. In the example below, the shapes show square corners, one shared radius, and a smaller top-left radius with a shared value for the remaining corners.

```
#grid(
  rows: 3,
  gutter: 12pt,
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 0pt,
    fill: aqua,
  ),
  squircle(
    width: 75pt,
    height: 48pt,
    radius: 35%,
    fill: aqua,
  ),
  squircle(
    width: 75pt,
    height: 48pt,
    radius: (top-left: 4pt, rest: 20pt),
    fill: aqua,
  ),
)
```

Inset and outset

`inset` pads the body, while `outset` expands the drawing without changing layout. In the example below, the first shape has neither, the second uses an inset, and the third uses an outset.

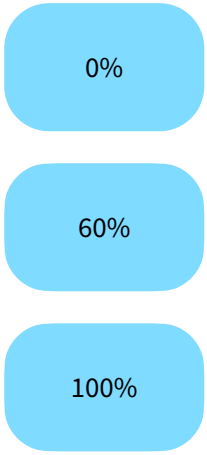
```
#grid(
  rows: 3,
  gutter: 12pt,
  squircle(
    width: 90pt,
    height: 48pt,
    radius: 10pt,
    fill: aqua,
  )[No inset or outset],
  squircle(
    width: 90pt,
    height: 48pt,
    inset: (x: 20pt, y: 10pt),
    radius: 10pt,
    fill: aqua,
  )[Inset adds padding],
  squircle(
    width: 90pt,
    height: 48pt,
    radius: 10pt,
    outset: 8pt,
    fill: aqua,
  )[Outset expands the drawing],
)
```



Smoothing

`smoothing` controls the gradual transition between edges and corners. In the example below, the shapes use `0%`, `60%`, and `100%` smoothing.

```
#grid(
  rows: 3,
  gutter: 12pt,
  ..(0%, 60%, 100%).map(s => align(center +
horizon)[
    #squircle(
      width: 75pt,
      height: 48pt,
      radius: 35%,
      smoothing: s,
      fill: aqua,
    )[#repr(s)]
  ]),
)
```



Preserve smoothing

`preserve-smoothing` decides how constrained corners handle excess smoothing. In the example below, `false` reduces smoothing, while `true` retains it and compresses the Bézier handles.

```
#grid(
  rows: 2,
  gutter: 12pt,
  ..(false, true).map(keep => align(center +
horizon)[
    #squircle(
      width: 75pt,
      height: 48pt,
      radius: 18pt,
      smoothing: 100%,
      preserve-smoothing: keep,
      fill: aqua,
    )[preserve-smoothing: #repr(keep)]
  ]),
)
```